```
from datetime import datetime
import random, time, math
from copy import deepcopy, copy
import decimal

class Board:
    def __init__(self, queen_count=8):
        self.queen_count = queen_count
        self.reset()

    def reset(self):
        self.queens = [-1 for i in range(0, self.queen_count)]

        for i in range(0, self.queen_count):
            self.queens[i] = random.randint(0, self.queen_count - 1)
            # self.queens[row] = column


    def calculateCost(self):
        threat = 0

        for queen in range(0, self.queen_count):
            for next_queen in range(queen+1, self.queen_count):
                if self.queens[queen] == self.queens[next_queen] or abs(queen - next_queen) == abs(self.queens[queen] - se
                    threat += 1

        return threat

    @staticmethod
    def calculateCostWithQueens(queens):
        threat = 0
        queen_count = len(queens)

        for queen in range(0, queen_count):
            for next_queen in range(queen+1, queen_count):
                if queens[queen] == queens[next_queen] or abs(queen - next_queen) == abs(queens[queen] - queens[next_queen]
                    threat += 1

        return threat

    @staticmethod
    def toString(queens):
        board_string = ""

        for row, col in enumerate(queens):
            board_string += "(%s, %s)\n" % (row, col)

        return board_string

    def getLowerCostBoard(self):
        displacement_count = 0
        temp_queens = self.queens
        lowest_cost = self.calculateCost(temp_queens)

        for i in range(0, self.queen_count):
            temp_queens[i] = (temp_queens[i] + 1) % (self.queen_count - 1)

            for j in range(queen+1, self.queen_count):
                temp_queens[j] = (temp_queens[j] + 1) % (self.queen_count - 1)

    def __str__(self):
        board_string = ""

        for row, col in enumerate(self.queens):
            board_string += "(%s, %s)\n" % (row, col)

        return board_string

class SimulatedAnnealing:
    def __init__(self, board):
        self.elapsedTime = 0;
        self.board = board
        self.temperature = 4000
        self.sch = 0.99
        self.startTime = datetime.now()


    def run(self):
        board = self.board
        board_queens = self.board.queens[:]
```

```
            solutionFound = False

            for k in range(0, 170000):
                self.temperature *= self.sch
                board.reset()
                successor_queens = board.queens[:]
                dw = Board.calculateCostWithQueens(successor_queens) - Board.calculateCostWithQueens(board_queens)
                exp = decimal.Decimal(decimal.Decimal(math.e) ** (decimal.Decimal(-dw) * decimal.Decimal(self.temperature)))

                if dw > 0 or random.uniform(0, 1) < exp:
                    board_queens = successor_queens[:]

                if Board.calculateCostWithQueens(board_queens) == 0:
                    print("Solution:")
                    print(Board.toString(board_queens))
                    self.elapsedTime = self.getElapsedTime()
                    print("Success, Elapsed Time: %sms" % (str(self.elapsedTime)))
                    solutionFound = True
                    break

            if solutionFound == False:
                self.elapsedTime = self.getElapsedTime()
                print("Unsuccessful, Elapsed Time: %sms" % (str(self.elapsedTime)))

            return self.elapsedTime

        def getElapsedTime(self):
            endTime = datetime.now()
            elapsedTime = (endTime - self.startTime).microseconds / 1000
            return elapsedTime


    if __name__ == '__main__':
        board = Board()
        print("Board:")
        print(board)
        SimulatedAnnealing(board).run()
```

```
Board:
(0, 0)
(1, 2)
(2, 3)
(3, 7)
(4, 3)
(5, 2)
(6, 3)
(7, 2)

Unsuccessful, Elapsed Time: 888.779ms
```

```
import itertools

def check_entailment_and_biconditional(expression_A, expression_B):
    """
    Evaluates three boolean variables A, B, C (or generic props)
    Checks if A entails B (A -> B is a tautology / always True when A is True)
    Prints a truth table and checks bidirectional equivalence (A <-> B)
    """
    variables = ['A', 'B', 'C']
    print(f"| { ' | '.join(variables) } | A (KB) | B (Query) | A => B | B => A | A <=> B | Entailed? |")
    print("|-" + "-|-".join(['-'*len(v) for v in variables]) + "-|--------|-----------|--------|--------|---------|---------

    all_entails = True
    all_biconditional = True

    for values in itertools.product([True, False], repeat=3):
        env = dict(zip(variables, values))

        # Unpack values for readability inside expressions
        A, B, C = env['A'], env['B'], env['C']

        # Define your logic for A (Knowledge Base) and B (Query) here:
        # Example: A entails B where A = (A and B), B = B (or customize)
        val_A = eval(expression_A, {}, env)
        val_B = eval(expression_B, {}, env)

        # Implications & Biconditional
        imp_A_to_B = (not val_A) or val_B
        imp_B_to_A = (not val_B) or val_A
        bicond = (val_A == val_B)
```

```
        # Entailment check: In all models where A is True, B must also be True
        row_entails = not val_A or val_B
        if val_A and not val_B:
            all_entails = False
        if not bicond:
            all_biconditional = False

        row_str = f"| {str(A):<5} | {str(B):<5} | {str(C):<5} | {str(val_A):<6} | {str(val_B):<9} | {str(imp_A_to_B):<6} | {
        # Adjust table format cleanly for A, B, C
        print(f"| {str(A):<5} | {str(B):<5} | {str(C):<5} | {str(val_A):<6} | {str(val_B):<9} | {str(imp_A_to_B):<6} | {str(

    print("\n--- Summary Results ---")
    print(f"Does A entail B ($A \\models B$)? : {all_entails}")
    print(f"Is it Bidirectional ($A \\equiv B$ / IFF)? : {all_biconditional}")

# Example usage:
# Let A be 'A and B' and B be 'B'
print("Evaluating: A = (A and B), B = B")
check_entailment_and_biconditional("A and B", "B")
```

```
Evaluating: A = (A and B), B = B
| A | B | C | A (KB) | B (Query) | A => B | B => A | A <=> B | Entailed? |
|---|---|---|--------|-----------|--------|--------|---------|-----------|
| True  | True  | True  | True   | True      | True   | True   | True    | True      |
| True  | True  | False | True   | True      | True   | True   | True    | True      |
| True  | False | True  | False  | False     | True   | True   | True    | True      |
| True  | False | False | False  | False     | True   | True   | True    | True      |
| False | True  | True  | False  | True      | True   | False  | False   | True      |
| False | True  | False | False  | True      | True   | False  | False   | True      |
| False | False | True  | False  | False     | True   | True   | True    | True      |
| False | False | False | False  | False     | True   | True   | True    | True      |

--- Summary Results ---
Does A entail B ($A \models B$)? : True
Is it Bidirectional ($A \equiv B$ / IFF)? : False
```